

Emotion Detection & Learning Support Engine

Project Description:

The **Emotion Detection & Learning Support Engine** is an AI-powered educational platform that analyzes students' emotional states from their textual inputs and provides personalized learning guidance. The system leverages advanced Natural Language Processing (NLP) techniques, deep learning models, and Generative AI to understand how students feel while studying and recommend appropriate learning support.

The platform integrates **BiLSTM** and **BERT** models for accurate emotion classification into five emotional categories: **Bored, Confident, Confused, Curious, and Frustrated**. Based on the detected emotion, the application utilizes the **Google Gemini API** to generate personalized learning suggestions, motivational guidance, and study strategies that help learners overcome challenges and improve their learning experience.

Built using **Streamlit**, the application provides an intuitive and interactive user interface where students can enter their learning concerns, instantly receive emotion predictions, visualize confidence scores, and obtain AI-generated educational support. The system also stores prediction records for future analysis and provides emotion analytics through interactive visualizations.

By combining deep learning-based emotion recognition with Generative AI, the Emotion Detection & Learning Support Engine creates an intelligent learning companion that enhances student engagement, supports emotional well-being, and promotes effective personalized education.

Scenarios

Scenario 1: Confused Student

A student enters the text:

"I have studied this topic many times, but I still cannot understand the concepts."

The system predicts the emotion as **Confused** using the BiLSTM and BERT models. Based on this prediction, Gemini AI generates step-by-step explanations, study techniques, and motivational guidance to help the student understand the topic more effectively.

Scenario 2: Frustrated Learner

A learner writes:

"No matter how much I practice, I keep making mistakes and I feel like giving up."

The platform detects the emotion Frustrated and generates encouraging responses, stress-management tips, revision strategies, and practical recommendations to rebuild confidence and maintain consistent learning.

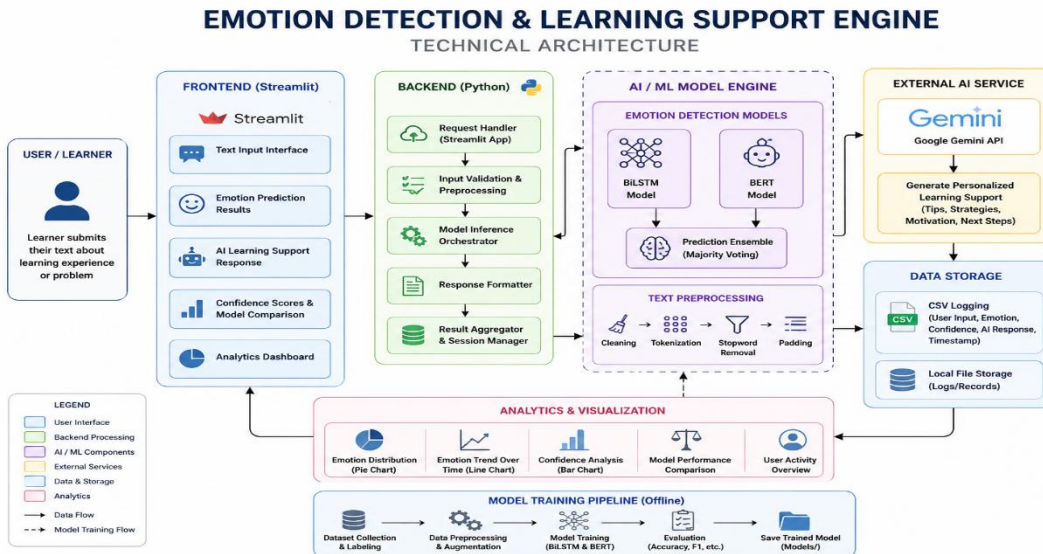
Scenario 3: Curious Student

A student submits:

"I enjoyed today's lesson and want to learn more about Artificial Intelligence."

The application identifies the emotion as **Curious** and recommends advanced learning resources, additional topics, project ideas, and personalized study paths to encourage continuous learning and exploration.

Technical Architecture:



Pre-requisites:

- Python Documentation: <https://docs.python.org/3/>
- TensorFlow Documentation: <https://www.tensorflow.org/>
- Hugging Face Transformers: <https://huggingface.co/docs/transformers>
- Streamlit Documentation: <https://docs.streamlit.io/>
- Google AI Studio (Gemini API): <https://aistudio.google.com/>
- Git Documentation: <https://git-scm.com/doc>
- Visual Studio Code: <https://code.visualstudio.com/>

Project Workflow:

Milestone 1: Environment Setup and Model Preparation

Activity 1.1

Set up the Python development environment by installing Python 3.11, creating a virtual environment, and installing all required libraries from requirements.txt.

Activity 1.2

Obtain and configure the Google Gemini API Key securely using environment variables (GOOGLE_API_KEY) for AI-generated personalized learning support.

Activity 1.3

Prepare the trained BiLSTM and BERT emotion detection models by placing them in the appropriate models/ directory and verifying their compatibility with the application.

Activity 1.4

Design the overall system architecture, defining interactions between the Streamlit frontend, Python backend, emotion detection models, Gemini AI, and CSV logging module.

Milestone 2: Emotion Detection Module Development

Activity 2.1

Develop the text preprocessing pipeline, including text cleaning, tokenization, stop-word removal, and input preparation.

Activity 2.2

Implement the BiLSTM and BERT emotion detection models to classify learner emotions and generate confidence scores.

Activity 2.3

Integrate model prediction logic and compare outputs from both models to improve emotion classification accuracy.

Milestone 3: AI Learning Support Development

Activity 3.1

Integrate the Google Gemini API to generate personalized learning guidance based on the detected emotion.

Activity 3.2

Develop response generation logic to provide study tips, motivational messages, learning strategies, and next-step recommendations.

Milestone 4: Streamlit Application Development

Activity 4.1

Develop the Streamlit user interface for entering learning-related text, displaying detected emotions, confidence scores, and AI-generated responses.

Activity 4.2

Implement interactive dashboards using Plotly to visualize emotion distribution, prediction confidence, and model performance.

Activity 4.3

Implement CSV logging to store user inputs, predicted emotions, AI responses, confidence scores, and timestamps for future analysis.

Milestone 5: Testing and Deployment

Activity 5.1

Perform functional testing, validate emotion prediction accuracy, verify Gemini AI integration, and ensure all application modules work correctly.

Activity 5.2

Configure the application for local execution using Streamlit, verify dependency installation, and prepare the project for future cloud deployment using Google Cloud Run.

Milestone 6: Conclusion

Activity 6.1

Prepare comprehensive project documentation, including system architecture, ER diagram, DFD, technology stack, implementation details, testing results, and user guide.

Activity 6.2

Summarize the project outcomes, discuss achieved objectives, highlight system advantages, identify limitations, and propose future enhancements such as multilingual support, voice-based emotion detection, and Learning Management System (LMS) integration.

Milestone 1: Model Selection and Architecture

In this milestone, we focus on preparing the AI models and overall system architecture for the Emotion Detection & Learning Support Engine. The application uses BiLSTM and BERT models to detect students' emotions from text input and integrates the Google Gemini API to generate personalized learning support. This milestone also includes configuring the development environment and defining the interaction between the Streamlit interface, backend processing, AI models, and data storage.

Step 1 — Open Google AI Studio

Visit:

<https://aistudio.google.com/>

Sign in using your Google account.

Step 2 — Navigate to API Keys

After logging in, click **API Keys** from the left navigation menu.

Step 3 — Create a New API Key

Click **Create API Key**.

Step 4 — Copy the API Key

Copy the generated API key immediately and store it securely.

Step 5 — Configure the API Key

Create a **.env** file in the project root directory and add:

GOOGLE_API_KEY=your_api_key_here

Step 6 — Verify the API Key

Run the application and ensure Gemini AI responses are generated successfully

Activity 1.2: Research and Select the Appropriate Generative AI Model

The project uses two deep learning models for emotion detection and one Generative AI model for personalized responses.

- **Understand Project Requirements**
- Study how students describe learning problems through text and identify the emotions that influence their learning experience.
- **Evaluate Emotion Detection Models**
- Compare multiple NLP models based on prediction accuracy, inference speed, and robustness.
- **Select the Emotion Detection Models**
- The following models were selected:
 - BiLSTM**
 - BERT**
- These models provide high emotion classification accuracy for educational text.
- **Select the Generative AI Model**
- The **Google Gemini API** was selected to generate personalized learning guidance, motivational responses, and study recommendations based on the detected emotion.

Activity 1.3: Define the Architecture of the Application

Draft the Architecture Diagram

Create an architectural diagram showing the interaction between:

- Streamlit User Interface
- Python Backend
- BiLSTM Model
- BERT Model
- Google Gemini API
- CSV Logging Module

Frontend Functionality

The Streamlit application allows users to:

- Enter learning-related text
- Submit academic concerns
- View detected emotions
- View confidence scores

- Receive AI-generated learning guidance
- Visualize analytics through charts

Backend Responsibilities

The Python backend is responsible for:

- Receiving user input
- Validating the text
- Performing text preprocessing
- Running BiLSTM and BERT inference
- Calculating confidence scores
- Calling the Gemini API
- Returning personalized responses
- Saving interaction history into CSV files

AI Integration

The application integrates three AI components:

- **BiLSTM** for emotion prediction
- **BERT** for emotion prediction
- **Google Gemini API** for personalized learning support

The backend combines the detected emotion with user input and sends it to Gemini AI to generate context-aware educational guidance.

Activity 1.4: Set Up the Development Environment

Install Python

Install Python 3.11 or later.

Create a Virtual Environment

```
python -m venv .venv
```

Activate it:

```
.venv\Scripts\activate
```

Install Required Libraries

```
pip install -r requirements.txt
```

Install Streamlit

```
pip install streamlit
```

Configure the API Key

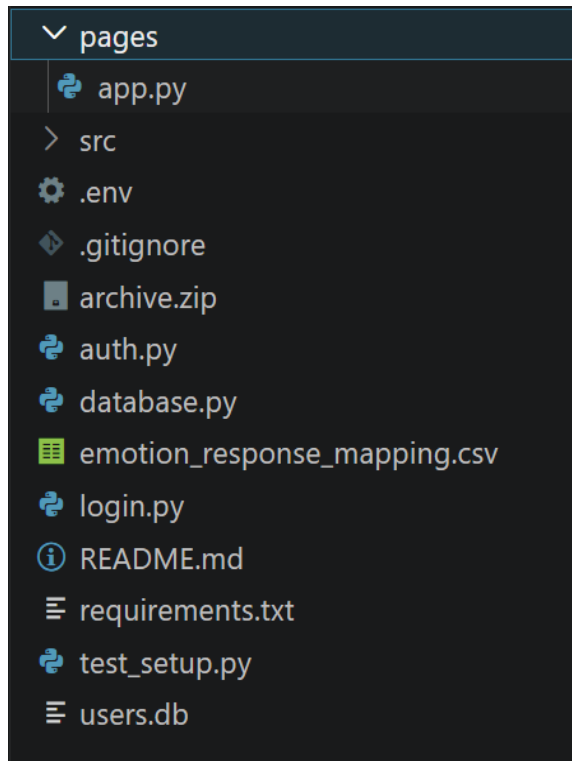
Create a .env file.

```
GOOGLE_API_KEY=your_api_key_here
```

Verify Model Files

Ensure the following folders exist:

```
models/  
├── bltsm/  
└── bert_emotion_model_final/
```



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Emotion Detection Features

Implement the three core functionalities of the application that work together to provide intelligent learning support.

Text Input Processing

- Allow users to enter learning-related text describing their academic challenges or study experiences.
- Perform text preprocessing, including cleaning, tokenization, and normalization, to prepare the input for emotion detection.

Emotion Detection

- Use the trained BiLSTM and BERT models to classify user input into one of the predefined emotion categories:
 - Bored
 - Confident
 - Confused
 - Curious
 - Frustrated
- Calculate confidence scores for the predicted emotion and compare predictions from both models to improve reliability.

Personalized Learning Support

- Send the detected emotion and processed user input to the Google Gemini API.

- Generate personalized learning guidance, motivational messages, study strategies, and recommendations based on the user's emotional state.

Activity 2.2: Implement the Streamlit Backend

Develop the Streamlit Application

- Build the application using Streamlit to provide an interactive and user-friendly interface.
- Organize the application into modular components for preprocessing, prediction, AI response generation, and visualization.

Process User Input

- Create a text input area where users can describe their learning problems.
- Validate that the input is not empty before processing.
- Pass the input through the text preprocessing pipeline before emotion prediction.

Integrate AI Models

- Load the trained **BiLSTM** and **BERT** models from the models/ directory.
- Perform emotion prediction using both models.
- Display the predicted emotion and confidence score to the user.

Integrate Google Gemini API

- Use the configured **Google Gemini API Key** to communicate with the Gemini model.
- Generate personalized learning support based on the detected emotion.
- Display AI-generated suggestions directly within the Streamlit interface.

Store Prediction Results

- Save user input, predicted emotion, confidence score, AI-generated response, and timestamp into CSV log files.
- Maintain prediction history for analytics and future evaluation.

Display Results

- Present emotion prediction results using a clean Streamlit interface.
- Display confidence scores, personalized guidance, and interactive visualizations such as emotion distribution and analytics charts using Plotly.

Milestone 3: Application Development (app.py)

Milestone 3 focuses on developing the main application logic of the **Emotion Detection & Learning Support Engine**. The app.py file serves as the central controller of the application, coordinating interactions between the Streamlit user interface, emotion detection models, Google Gemini AI, and analytics components.

Activity 3.1: Develop the Main Application Logic

- Developed the main Streamlit application (app.py) to provide an interactive interface for users.
- Implemented user input validation to ensure meaningful learning-related text is submitted.
- Integrated the **BiLSTM** and **BERT** models for emotion prediction.
- Configured the **Google Gemini API** to generate personalized learning guidance based on the detected emotion.
- Displayed emotion prediction results, confidence scores, and AI-generated responses in the Streamlit interface.
- Implemented CSV logging to store user interactions and prediction history.

- Added interactive analytics and visualization using Plotly.
- Implemented exception handling to ensure stable application performance and meaningful error messages.

Outcome

The app.py module successfully integrates all major components of the application, enabling users to enter learning-related text, detect emotions, receive personalized AI-generated guidance, and visualize prediction results through an interactive Streamlit dashboard.

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing a user-friendly and interactive interface for the **Emotion Detection & Learning Support Engine**. The frontend is developed using **Streamlit**, providing a clean and responsive interface that allows users to enter learning-related text, view emotion prediction results, receive personalized learning guidance, and visualize analytics.

Activity 4.1: Designing and Developing the User Interface

Develop the Streamlit Interface

- Design the main application using Streamlit with a simple and intuitive layout.
- Create a homepage containing the project title, description, text input area, prediction button, and results section.
- Organize the interface using Streamlit containers, columns, headers, and expandable sections for better readability.

Create a Responsive Layout

- Develop a responsive interface that works smoothly on desktop and laptop screens.
- Apply custom styling using Streamlit themes and CSS where required to improve the application's appearance.
- Ensure consistent fonts, colors, spacing, and layout throughout the application.

Design User Interface Components

The application includes the following user interface components:

- **Text Input Section:** Allows users to enter learning-related problems or study experiences.
- **Prediction Section:** Displays the detected emotion and confidence score generated by the BiLSTM and BERT models.
- **AI Response Section:** Shows personalized learning guidance and motivational support generated by the Google Gemini API.
- **Analytics Dashboard:** Displays interactive charts and visualizations using Plotly.
- **History Section:** Presents previously recorded predictions and user interaction logs.

Activity 4.2: Displaying Dynamic Results

Process User Input

- Accept user input through the Streamlit interface.
- Validate the input before sending it to the emotion detection models.
- Display warning messages if the input is empty or invalid.

Display Emotion Prediction

- Show the predicted emotion along with its confidence score.

- Present predictions generated by both BiLSTM and BERT models in a clear format.

Display AI-Generated Learning Support

- Display personalized study recommendations, motivational messages, and learning strategies generated by Google Gemini AI.
- Format the responses for easy readability.

Display Analytics

- Present emotion distribution, prediction confidence, and model performance using interactive Plotly charts.
- Update the visualizations dynamically whenever new predictions are generated.

Refresh User Interface

- Automatically update the displayed results after each prediction.
- Clear previous loading indicators and ensure the interface remains responsive during processing.

Outcome

At the completion of this milestone, the Streamlit-based frontend provides an intuitive, responsive, and interactive user experience. Users can easily submit learning-related text, receive accurate emotion predictions, view AI-generated guidance, and explore analytics through a clean and well-organized dashboard.

Milestone 5: Deployment

Milestone 5 focuses on preparing the **Emotion Detection & Learning Support Engine** for execution in the local development environment. This milestone includes configuring the Python environment, installing project dependencies, verifying the Google Gemini API configuration, and running the Streamlit application successfully.

Activity 5.1: Preparing the Application for Local Deployment

Set Up the Development Environment

- Open the project folder in **Visual Studio Code**.
- Open a new terminal from **Terminal** → **New Terminal**.
- Create and activate a Python virtual environment.

```
python -m venv .venv
```

Windows:

```
.venv\Scripts\activate
```

Install Project Dependencies

Install all required libraries using the requirements file.

```
pip install -r requirements.txt
```

Configure Environment Variables

Create a .env file in the project root directory.

Add the Google Gemini API key:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

The application automatically reads the API key during startup.

Verify Project Resources

Ensure the following resources are available before running the application:

- Trained BiLSTM model
- Trained BERT model
- Dataset files
- Required Python libraries
- Google Gemini API Key

Activity 5.2: Local Testing and Verification

Run the Streamlit Application

Start the application using the following command:

```
streamlit run app.py
```

Access the Application

Once the application starts successfully, open the browser and navigate to:

<http://localhost:8501>

Verify Application Functionality

Test the following modules:

- User text input
- Emotion prediction using BiLSTM
- Emotion prediction using BERT
- Confidence score generation
- Google Gemini AI response generation
- Analytics dashboard

- CSV logging functionality

Verify that all modules work correctly and produce the expected results.

Activity 5.3: Deployment Readiness

The application has been prepared for deployment by:

- Configuring the Python virtual environment.
- Installing all required project dependencies.
- Securing the Google Gemini API key using a .env file.
- Organizing project files into a modular folder structure.
- Successfully validating all application functionalities through local testing.

The project currently runs successfully in the local development environment using Streamlit and can be deployed to cloud platforms such as **Google Cloud Run**, **Streamlit Community Cloud**, or **Render** in the future if required.

Important Notes

- Always activate the virtual environment before running the application.
- Ensure all dependencies from requirements.txt are installed.
- Never upload the .env file or API key to GitHub.
- Verify that the trained model files are available inside the models directory.
- Maintain a stable internet connection for Google Gemini API communication.
- Run the application using:

```
streamlit run app.py
```

for proper execution.

Exploring the Website's Web Pages:

Login Page:

The Login Page serves as the entry point to the Emotion Detection & Learning Support Engine. It provides a secure authentication system where existing users can log in using their email and password, while new users can create an account through the registration option. Upon successful authentication, users are redirected to the main application, where they can access emotion detection, personalized learning support,

and interactive analytics. The page features a simple, user-friendly interface designed to ensure a smooth and secure login experience.



Emotion Detection & Learning Support Engine

Login to continue

☒ Login ☐ Register

Email

demo123@gmail.com

Password

1234567 

Login

Home / Generator Page:

The Analyze Emotion page is the main interface of the Emotion Detection & Learning Support Engine. It allows users to enter their learning challenges, detect emotions using AI models, and receive personalized study guidance and recommendations through an interactive Streamlit dashboard.

 Stop Deploy



Emotion Detection & Learning Support Engine

Understand your learning obstacles, analyze sentiment with our BiLSTM classifier, and access targeted study strategies.

 Analyze Emotion

 Session History & Analytics

 Neural Network Diagnostics



Input Your Challenge

Select Study Field

Computer Science

Describe what you are working on, what is causing difficulty, or how you feel about it:

Example: I'm trying to learn recursion, but my functions keep hitting stack overflows. I'm completely stuck...



Click to Test Templates

Select a preset challenge matching target emotional states:

 Confused - Recursion base cases

 Curious - Focal Loss vs Cross-Entropy

 Frustrated - 3-hour segfault debug

 Confident - Successful validation run

Input Section:

The Input Section allows users to select their study field and describe their learning challenges or academic difficulties. The provided information is used by the AI models to analyze emotions and generate personalized learning support.

Example: I'm trying to learn recursion, but my functions keep hitting stack overflows. I'm completely stuck...

Analyze Emotion

Frustrated - 3-hour segfault debug

Confident - Successful validation run

Note: The classifying model is trained on student expressions. Including emotion phrases like 'I feel confused' or 'I am frustrated' helps the NLP engine deliver higher accuracy predictions.

Results Section:

Analysis Output

The Analysis Output section displays the detected emotion along with personalized learning recommendations generated by the AI based on the user's input.

Detected Emotion

This section shows the predicted emotional state of the user along with a brief explanation of the detected emotion.

Personalized Recommendation

This section provides AI-generated study guidance and motivational recommendations tailored to the detected emotion.

Classification Confidence

This section displays the confidence score of the emotion prediction, indicating the reliability of the classification.

State Probabilities

This section visualizes the probability distribution for each emotion, helping users understand the model's prediction confidence across all emotion categories.

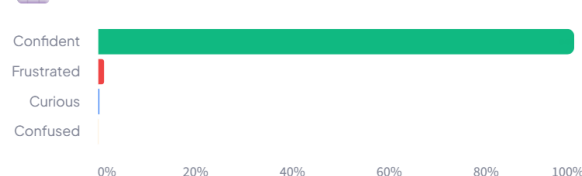
Recommended Study Strategies

This section suggests personalized study techniques and learning strategies based on the detected emotional state to improve learning outcomes.

Classification Confidence



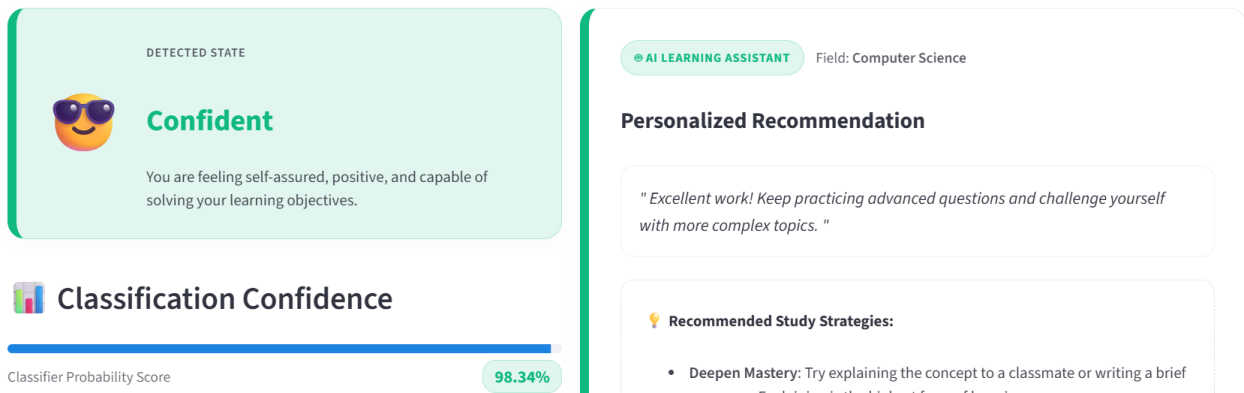
State Probabilities



Recommended Study Strategies:

- **Deepen Mastery:** Try explaining the concept to a classmate or writing a brief summary. Explaining is the highest form of learning.
- **Extend Scope:** Challenge yourself with edge-cases, code optimization, or advanced projects based on the topic.
- **Build & Deploy:** Apply the knowledge to a sandbox project to ensure long-term retrieval.

Analysis Output

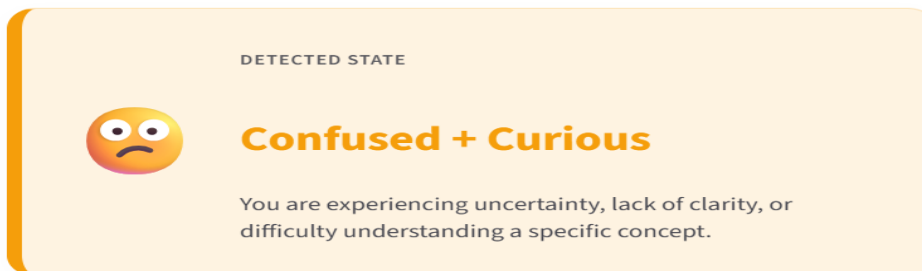


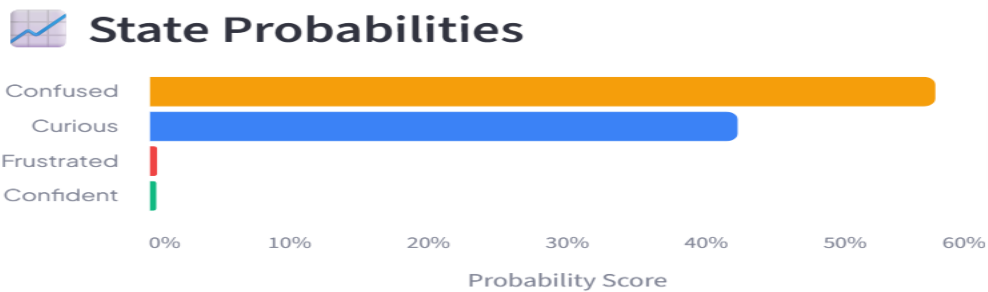
Mixed emotion:

Input: I understand the basics, but I'm still confused about recursion and also curious to learn more.

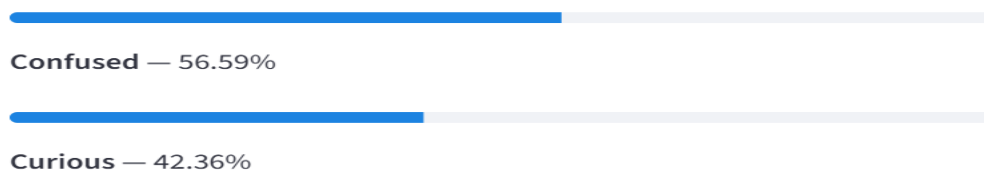
Output:

Analysis Output





Mixed Emotion Detection



Conclusion:

The **Emotion Detection & Learning Support Engine** is an AI-powered educational platform developed using **Streamlit**, **BiLSTM**, **BERT**, and the **Google Gemini API** to provide personalized learning support based on students' emotional states. By analyzing learners' text input, the system accurately identifies emotions such as **Bored**, **Confident**, **Confused**, **Curious**, and **Frustrated**, and generates customized study recommendations, motivational guidance, and learning strategies. The project successfully integrates deep learning, natural language processing, and generative AI into a single interactive application, helping students improve both their emotional well-being and learning outcomes.

Looking ahead, the platform offers significant opportunities for enhancement, including multilingual support, voice-based emotion detection, integration with Learning Management Systems (LMS), personalized learning progress tracking, and cloud deployment for wider accessibility. With continued improvements in AI models and user experience, the Emotion Detection & Learning Support Engine has the potential to become a comprehensive intelligent learning assistant for students across various educational domains.